

SMARTCASHIER

PANDUAN REPLIKASI PROYEK DARI NOL

Mata Kuliah: Pemrograman Berorientasi Objek

Kelompok: 7

Anggota Kelompok:

1. Farel Gusviransyah
2. Lutfi Nasrullah Aziz
3. Muhammad Alfin Ramadhan
4. Muhammad Fadli Al Hafizh Wibiksana
5. Ringo Noer Junaedy

Di-generate secara otomatis untuk Kelompok 7

PANDUAN REPLIKASI PROYEK SMARTCASHIER DARI NOL

Panduan ini berisi petunjuk langkah-demi-langkah (step-by-step) yang komprehensif dan mendetail untuk mereplikasi dan membangun ulang proyek **SMARTCASHIER** dari awal. Diasumsikan bahwa JDK 17, Node.js (v18+), NPM, dan PostgreSQL sudah terinstal pada komputer Anda.

STRUKTUR LAYOUT PROYEK

Buat struktur direktori proyek seperti di bawah ini:

```
pbo-tubes-kelompok7/
```

schem

LANGKAH 1: KONFIGURASI DATABASE

1. Masuk ke PostgreSQL Anda (via pgAdmin atau psql CLI).

2. Jal

-- schema.sql

CREATE

CREATE TABLE member (

10

CREATE TABLE product (

10

CREATE TABLE transaction (

10

```
CREATE TABLE transaction_item (
```

```
-- Data Seeding
```

```
INSERT INTO member (id, name, phone_number, join_date, discount_rate) VALUES
```

```
INSERT INTO product (product_name, category, quantity, price) VALUES
```

```
---
```

LANGKAH 2: MEMBANGUN BACKEND (JAVA SPRING BOOT)

1. File Konfigurasi Maven (`pom.xml`)

Buat file `backend/pom.xml` dengan isi berikut:

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<groupId>com.smartcashier</groupId>
```

```
<properties>
```

```
<dependencies>
```

```
<build>
```

2. Properties Aplikasi (`application.properties`)

Buat file `backend/src/main/resources/application.properties`:

```
spring.application.name=smartcashier
```

```
spring.datasource.url=jdbc:postgresql://localhost:5432/smartcashier_db
```

```
spring.jpa.hibernate.ddl-auto=update
```

```
spring.jackson.time-zone=Asia/Jakarta
```

3. Implementasi Classes Entity (com.smartcashier.entity)

- **User.java (Abstract Class)**

```
package com.smartcashier.entity;
```

```
import jakarta.persistence.MappedSuperclass;
```

```
@MappedSuperclass
```

```
public User() {}
```

```
public User(String id, String name, String phoneNumber) {
```

```
public abstract void displayProfile();
```

- ****Karyawan.java****

```
package com.smartcashier.entity;
```

```
import jakarta.persistence.Entity;
```

```
@Entity
```

```
public Karyawan() {}
```

```
public Karyawan(String id, String name, String phoneNumber, String password) {
```

```
public boolean login(String inputId, String inputPassword) {
```

```
public void addKpi(double score) {
```

```
@Override
```

- ****Member.java****

```
package com.smartcashier.entity;
```

```
import jakarta.persistence.Entity;
```

```
@Entity
```

```
public Member() {}
```

```
public Member(String id, String name, String phoneNumber, String joinDate, double discountRate) {
```

```
public double applyDiscount(double subtotal) {
```

```
@Override
```

- ****Product.java****

```
package com.smartcashier.entity;
```

```
import jakarta.persistence.*;
```

```
@Entity
```

```
private String productName;
```

- ****Transaction.java****

```
package com.smartcashier.entity;
```

```
import com.smartcashier.service.CalculatorTools;
```

```
@Entity
```

```
private String transactionId;
```

```
@ManyToOne
```

```
@ManyToOne
```

```
private Double totalAmount;
```

```
@OneToMany(mappedBy = "transaction", cascade = CascadeType.ALL, orphanRemoval = true)
```

```
public void calculateTotal() {
```

```
    if (member != null) {
```

- ****TransactionItem.java****

```
package com.smartcashier.entity;
```

```
import com.fasterxml.jackson.annotation.JsonIgnore;
```

```
@Entity
```

```
@ManyToMany
```

```
@ManyToMany
```

```
private Integer quantity;
```

4. Implementasi Jpa Repositories (com.smartcashier.repository)

Buat interface JPA Repository standar untuk masing-masing Entity:

5. Implementasi Services (com.smartcashier.service)

- **CalculatorTools.java** (Utility Perhitungan Matematika)**

```
package com.smartcashier.service;
```

```
public class CalculatorTools {
```

- **TransactionService.java** (Pusat Logika Transaksi)**

```
package com.smartcashier.service;
```

```
import com.smartcashier.dto.ReceiptResponse;
```

```
@Service
```

```
@Transactional
```

```
Transaction transaction = new Transaction();
```

```
if (request.getMemberPhone() != null && !request.getMemberPhone().trim().isEmpty()) {
```

```
for (TransactionRequest.ItemRequest ir : request.getItems()) {
```

```
    productService.decreaseStock(product.getId(), ir.getQuantity());
```

```
    TransactionItem item = new TransactionItem();
```

```
        transaction.getItems().add(item);
```

```
transaction.calculateTotal();
```

```
Transaction saved = transactionRepository.save(transaction);
```

```
public ReceiptResponse getReceipt(String transactionId) {
```

```
private ReceiptResponse mapToReceipt(Transaction tx) {
```

- ****ProductService.java (Stock Safe Update)****

```
package com.smartcashier.service;
```

```
import com.smartcashier.entity.Product;
```

```
@Service
```

```
public synchronized void decreaseStock(Integer productId, int quantity) {
```

- Tambahkan juga `KaryawanService.java` & `MemberService.java` untuk melayani pengambilan data profil karyawan dan pendaftaran member baru.

6. Implementasi REST Controllers (com.smartcashier.controller)

Buat Controller dengan mapping REST API standar:

LANGKAH 3: MEMBANGUN FRONTEND (REACT.JS + VITE)

1. package.json & vite.config.js

- Inisialisasi Vite menggunakan shell command:

```
import { defineConfig } from 'vite'

export default defineConfig({
```

2. Styling Global (`src/index.css`)

Tambahkan utility UI bernuansa Dark Modern & Glassmorphism di `index.css`:

```
:root {

body {

.glass-panel {

/* Button & Form styles */
```

```
.form-input {
```

```
width
```

```
/* Print Layout rules */
```

```
@media
```

3. Setup Axios Client (`src/services/api.js`)

```
import axios from 'axios';
```

```
const api = axios.create({
```

```
base
```

```
api.interceptors.request.use((config) => {
```

```
cons
```

```
export default api;
```

4. Implementasi Halaman Dashboard & Cart (`src/components/`)

- **Login.jsx**: Form autentikasi. Menghubungi POST `/api/auth/login`. Simpan data respon (`id`, `name`, `kpiScore`) ke `localStorage.setItem('karyawan', ...)` dan panggil callback masuk.

- **[...]**
(`P`

LANGKAH 4: PROSES MENJALANKAN APLIKASI

1. **Jalankan Database**: Pastikan PostgreSQL Service aktif dan port 5432 terbuka.

2. **Jalankan Aplikasi**